

CLUSTER VALIDATION INDICES AS SELF-SUPERVISED OBJECTIVES FOR TEXT REPRESENTATION LEARNING

Kishor Kumar Bhaumik¹ Nicolas Roque dos Santos¹ Neil Shah²
Jia Chen¹ Evangelos E. Papalexakis¹

¹University of California, Riverside ²Pinterest

ABSTRACT

Self-supervised fine-tuning refines the embedding space of a pretrained language encoder without labels. However, the commonly used approaches are computationally expensive. Specifically, contrastive learning-based methods need multiview data and in-batch negative examples, while negative-free approaches require auxiliary graphs/networks. An interesting question arises: can self-supervised fine-tuning be done without relying on either additional negatives or graph data? To answer this question, we introduce SILK (Silhouette-guided K-means), which trains on a Cluster Validation Index, an internal measure of cluster quality without using labels. SILK clusters the corpus and then regresses a simplified silhouette toward a target value. Each document is compared only against the k cluster centroids, never against other documents, so the method needs no augmentation, no negative pairs and one view per document. On BERT-base, SILK trains $1.46\times$ faster per epoch than the fastest baseline we evaluate and uses 45.4% less peak GPU memory than the leanest one. Under frozen-encoder linear probing, SILK stays competitive with the best baselines on three downstream tasks. Our code is available at <https://github.com/Kishor-Bhaumik/SILK>.

Index Terms— Self-supervised learning, cluster validation index, silhouette, BERT, representation learning.

1. INTRODUCTION AND RELATED WORK

Text embeddings underpin retrieval, semantic search, clustering, and retrieval-augmented generation, where the evidence supplied to a language model is selected by the geometry of the embedding space [1]. Pretrained encoders such as BERT [2] carry rich contextual information, but masked language modeling, in general, does not force text embeddings to align with geometrical information, i.e., the resulting space is anisotropic and separates latent semantic categories poorly [3]. Self-supervised fine-tuning addresses this by reshaping the space with unlabeled text.

Contrastive approaches such as SimCSE [4] and DfCSE [5] build two views of each sentence via dropout masks

or edit-based auxiliary tasks and contrast them against in-batch negatives, with later work [6] refining the augmentations and the sampling. Introducing negatives adds computational cost: pairwise comparison is quadratic in the batch and semantically related texts drawn as negatives are actively pushed apart. Negative-free methods instead prevent collapse architecturally, through feature decorrelation, asymmetry, or momentum encoders [7]; SCD [8] combines a self-contrastive term with a decorrelation penalty over two dropout views. Collapse avoidance then becomes an extra component to design and tune.

This suggests a question that predates deep representation learning: *can a classical clustering criterion serve directly as the self-supervised objective?* Cluster Validation Indices (CVIs) score a partition by intra-cluster compactness and inter-cluster separation with no ground truth. Among these indices, the silhouette index [9] encodes exactly the preference contrastive learning expresses through pairs; same-cluster points close, different-cluster points far—but does so without constructing any pairs. CVIs have been used for five decades to *evaluate* clusterings. Using a CVI itself as a differentiable training loss is recent [10].

In the literature, clustering results have been used to guide training neural networks in self-supervised settings. For instance, DeepCluster [11] trains a CNN on k -means pseudolabels and graph methods use clustering to select positives or to optimize cluster quality directly [10]. CARL-G [10] trains a GNN with a simplified-silhouette loss, and uses a transductive GCN over a fixed graph generating all node embeddings in a single forward pass. However, neither method is directly applicable to text corpora as documents are independent sequences without explicit graph structure and the activation memory of a Transformer makes a full-corpus computation graph infeasible for corpora of any size. Therefore, it is worth investigating whether CVI-based learning transfers to text data.

In light of this, we propose SILK (Silhouette-guided K-means). SILK first embeds an unlabeled corpus of n documents and applies k -means to obtain corpus-level centroids. It then optimizes a simplified silhouette, which replaces dis-

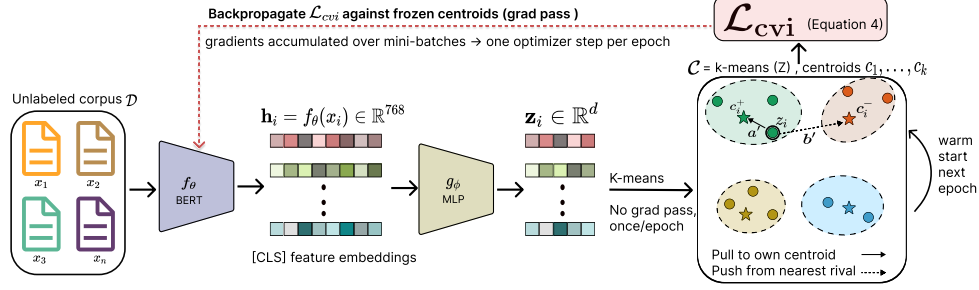


Fig. 1. Overview of SILK. Documents $x_1, \dots, x_n$ are encoded by BERT and projected. k -means is fit on the resulting embeddings and the silhouette loss pulls each document toward its assigned centroid while pushing it from the nearest competing one. That CVI loss is backpropagated into BERT. Unlike contrastive methods, no augmented views or negative pairs are constructed.

tances to other documents by distances to the k centroids. The computational cost therefore drops from $\mathcal{O}(n^2)$ pairwise distances to $\mathcal{O}(nk)$ embedding-to-centroid distances, with $k \ll n$. To run this on a Transformer we separate the scope of the clustering from the scope of the computation graph, refreshing centroids once per epoch and accumulating gradients across batches into a single optimizer step. Our contributions are summarized below.

- **CVI loss.** SILK optimizes cluster quality directly, contrasting each document against k corpus-level centroids instead of against other documents. It uses one view per document and requires no augmented views or negative pairs.
- **Feasible training scheme.** A silhouette loss needs the whole corpus clustered but a Transformer can backpropagate only one batch at a time. We cluster the corpus once per epoch and accumulate gradients across batches, so peak activation memory scales with the batch rather than with the corpus.
- **Lower training cost.** With a single view and no in-batch negatives, per-epoch time and peak GPU memory fall below every baseline we evaluate, while linear-probe accuracy stays competitive with the strongest.

2. PROPOSED METHOD

Figure 1 shows the method. Documents are encoded by BERT, projected, and clustered with k -means. The CVI loss on that clustering is backpropagated to fine-tune the encoder.

2.1. Setup

Given an unlabeled corpus $\mathcal{D} = \{x_1, \dots, x_n\}$ of n documents, let f_θ be BERT-base with [CLS] pooling and $g_\phi : \mathbb{R}^{768} \rightarrow \mathbb{R}^d$ a 2-layer MLP projector with $d \ll 768$. Let $\mathbf{h}_i = f_\theta(x_i)$ be the encoder output. The embedding of document i is

$$\mathbf{z}_i = \frac{g_\phi(f_\theta(x_i))}{\|g_\phi(f_\theta(x_i))\|_2}, \quad i = 1, \dots, n, \quad (1)$$

Normalization keeps a few high-variance directions from dominating the distances, which matters because BERT features are anisotropic [3]. The projector is used only during fine-tuning. At evaluation we discard g_ϕ and use the 768-dimensional encoder features [7, 10]. Let $\mathcal{C} = \{\mathcal{C}_1, \dots, \mathcal{C}_k\}$ be a k -means partition of $\{\mathbf{z}_i\}_{i=1}^n$, $\mathbf{c}_j$ the centroid of $\mathcal{C}_j$, $\mathbf{c}_i^+$ the centroid of the cluster containing $\mathbf{z}_i$ and $\mathbf{c}_i^-$ the nearest other centroid.

2.2. Silhouette-Guided Objective

The silhouette index [9] asks, for each point, whether it lies closer to its own cluster than to the nearest competing one, comparing the mean distance to fellow members (compactness) against the mean distance to the nearest other cluster (separation). This is the geometric preference contrastive learning encodes through pairs, obtained without pairs or labels. The computational cost grows quadratically with the number of documents, $\mathcal{O}(n^2)$, because all pairwise distances must be calculated and retained for backpropagation, which is untenable for a Transformer on tens of thousands of documents.

We therefore use the *simplified silhouette* [12, 10], which replaces each set of points by the centroid summarizing it:

$$a'(\mathbf{z}_i) = \|\mathbf{z}_i - \mathbf{c}_i^+\|_2, \quad b'(\mathbf{z}_i) = \|\mathbf{z}_i - \mathbf{c}_i^-\|_2, \quad (2)$$

where a' is how far a document has drifted from its own centroid and b' how close it has come to its nearest rival. The per-document score is

$$s'(\mathbf{z}_i) = \frac{b'(\mathbf{z}_i) - a'(\mathbf{z}_i)}{\max\{a'(\mathbf{z}_i), b'(\mathbf{z}_i)\}} \in [-1, 1], \quad (3)$$

near 1 when the document sits far nearer its own centroid than any other and negative when it would be better placed elsewhere. Each document is compared against k centroids rather than $n - 1$ neighbours, giving $\mathcal{O}(nkd)$ time and $\mathcal{O}(nk)$ memory.

What does optimizing Eq. (3) do to the encoder? In the common case $a' < b'$ we have $s' = 1 - a'/b'$, whose partials are $\partial s'/\partial a' = -1/b'$ and $\partial s'/\partial b' = a'/b'^2$. Raising the score therefore pulls the document toward its own centroid and pushes it from the nearest competing one. The push is weighted by a'/b'^2 , so a document already tight on its centroid is repelled weakly, while one caught between two clusters is repelled strongly. The objective contrasts each document against a small set of corpus-level *summaries* rather than against individual documents, which is precisely why no negative sampling is required.

We do not maximize the mean score. A silhouette of 1 is reached only when every cluster collapses to a point; since k -means inevitably misassigns some documents, maximization drives those onto the wrong centroid and destroys the within-cluster variation a downstream classifier needs [10]. We regress toward a bounded target $\tau \in [-1, 1]$ where the outer sum runs over the N_B mini-batches of an epoch:

$$\mathcal{L}_{\text{cvi}} = \frac{1}{N_B} \sum_B \left| \tau - \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} s'(\mathbf{z}_i) \right|, \quad (4)$$

2.3. Clustering under a Bounded Computation Graph

CARL-G evaluates its objective in one forward pass per epoch [10], which a transductive GCN permits because all n node embeddings live in a single computation graph. However, Transformer activations scale as $\mathcal{O}(nLTd)$ in documents, layers, tokens, and width, placing a full-corpus graph orders of magnitude beyond device memory. Stepping per mini-batch is no remedy either: k -means would then be fit to a few hundred randomly drawn documents, leaving the centroids of Eq. (2) sample-dependent and the loss batch-local.

We decouple the two scopes. An inference-mode pass builds no graph and produces the embeddings for GPU k -means [13], initialized with k -means++ [14] in the first epoch and warm-started from the previous centroids thereafter, so the loss target does not jump when the representation has barely moved. A second pass recomputes embeddings with gradients enabled, evaluates Eq. (4) against the frozen centroids, and frees each batch’s graph before building the next; the accumulated gradients give one optimizer step per epoch. Every batch is scored against the same frozen centroids, so this step is computed under one consistent clustering, while peak activation memory falls from $\mathcal{O}(n)$ to $\mathcal{O}(|\mathcal{B}|)$. We pay one extra forward pass and the graph shrinks by a factor of $n/|\mathcal{B}|$ where $|\mathcal{B}|$ is the batch size. This is what lets a silhouette objective run on a Transformer.

3. EXPERIMENTS

3.1. Setup

Protocol. All methods fine-tune `bert-base-uncased` on the unlabeled 20 Newsgroups [15] *all* subset ($\sim 18,846$ docu-

	SimCSE	DiffCSE	SCD	SILK
Avoids negative pairs	✗	✗	✓	✓
Augmentation-free	✗	✗	✗	✓
Single view per document	✗	✗	✗	✓
No extra anti-collapse term	✓	✗	✗	✓

Table 1. What each objective requires. SILK satisfies all four properties by contrasting against centroids instead of documents.

Method	TREC-6	Snippets	StackOverflow
BERT	79.04 ± 0.40	71.13 ± 0.50	72.76 ± 0.30
SimCSE	<u>90.64 ± 0.30</u>	<u>80.70 ± 0.40</u>	79.16 ± 0.50
DiffCSE	88.15 ± 0.60	80.67 ± 0.30	76.18 ± 0.40
SCD	89.15 ± 0.50	79.35 ± 0.40	77.28 ± 0.60
SILK (ours)	92.47 ± 0.40	82.04 ± 0.30	<u>78.24 ± 0.50</u>

Table 2. Linear-probe test accuracy (% , mean $\pm$ std over 5 seeds, each covering both fine-tuning and probing) on frozen features. All methods fine-tuned on 20 Newsgroups.

ments), with headers, footers and quotes removed. We fine-tune on documents rather than sentences so the corpus carries the topical structure a clustering objective can exploit. Afterwards we discard the projector, freeze the encoder and fit a linear classifier on its `[CLS]` features. No supervised gradient reaches the encoder, so the probe measures the geometry of the learned space rather than the capacity of downstream fine-tuning.

Baselines and tasks. We compare against raw BERT (no fine-tuning), SimCSE [4] and DiffCSE [5] (contrastive), and SCD [8] (negative-free, non-contrastive). We probe on TREC-6 [16] (6 classes), Snippets [17] (8 classes), and StackOverflow [18] (20 classes). These span question types, web snippets, and technical short text, and none matches the fine-tuning domain, so the probe also measures transfer. All methods, including the baselines, are fine-tuned for 20 epochs.

Hyperparameters. SILK uses target $\tau = 0.5$, a 2-layer MLP projector with output $d = 128$, and a maximum sequence length of 128 tokens. The batch size is 128. We train with AdamW at learning rate 10^{-5} and weight decay 10^{-4} for 20 epochs under a cosine schedule with 10 warmup epochs. GPU k -means runs for at most 100 assignment and update steps and stops earlier if it converges. The first epoch tries 10 initializations; later epochs warm-start from the previous centroids and use a single trial. We report the mean and standard deviation over 5 seeds, each covering both fine-tuning and linear probing. The number of clusters is chosen per downstream dataset by validation-split linear-probe accuracy ($k = 150, 120$, and 200 ; see Sec. 3.4). The probe trains a single linear layer with AdamW (lr 10^{-2} , weight decay 10^{-4} , batch 512) for 15 epochs, and we report final-epoch test accuracy.

Method	Time/epoch (min)	Peak GPU mem (GB)
SimCSE	3.017	21.823
DiffCSE	4.200	33.766
SCD	2.578	22.536
SILK	1.766	11.917

Table 3. Self-supervised training cost per epoch on an RTX A6000, batch size 128, FP32. Time includes the corpus embedding pass and k -means as well as the gradient update. Memory is the mean per-epoch peak of the PyTorch allocator; linear probing is excluded. All methods run for the same number of epochs (20), so per-epoch cost is proportional to total cost.

3.2. Downstream Accuracy

Table 2 reports linear-probe accuracy. SILK improves over the strongest baseline by 1.83 points on TREC-6 and 1.34 on Snippets, and trails SimCSE by 0.92 on StackOverflow. Every fine-tuning method improves substantially on raw BERT, confirming that the pretrained space is poorly suited to linear separation and that the gains come from reshaping it rather than from the probe. The StackOverflow result is the informative one. It has 20 classes of technical short text, the furthest of the three from the Usenet discussion corpus SILK was fine-tuned on, and it is also where a clustering objective has least to work with: with few tokens per document, the centroids defining a' and b' are estimated from sparse, overlapping topics. A loss built on cluster structure should give ground where that structure is weak, and it does. This also marks the boundary of where the method applies.

3.3. Training Cost

Table 3 reports per-epoch time and peak memory under matched conditions. SILK trains $1.46\times$ faster per epoch than SCD, the fastest baseline, and uses 45.4% less memory than SimCSE, the leanest—despite spending an extra forward pass on clustering every epoch. The saving is structural rather than incidental. Contrastive baselines hold two views of every document in the graph and materialize a $|\mathcal{B}| \times |\mathcal{B}|$ similarity matrix, so their negatives live *inside* the batch. SILK keeps one view and contrasts against k centroids that are external to the batch and fixed for the epoch. This is also why the two respond differently to batching: shrinking the batch alters the InfoNCE [19] objective by removing negatives, whereas for SILK it changes only how finely the same corpus-level gradient is accumulated.

3.4. Number of Clusters

A natural guess is to set k to the number of downstream classes. We instead treat k as a hyperparameter and, for each downstream dataset, train separate models across a range of k . Figure 2 plots validation-split linear-probe accuracy for that search, and Table 2 uses the k at each peak. Accuracy climbs steeply well past the class count, and on all three tasks

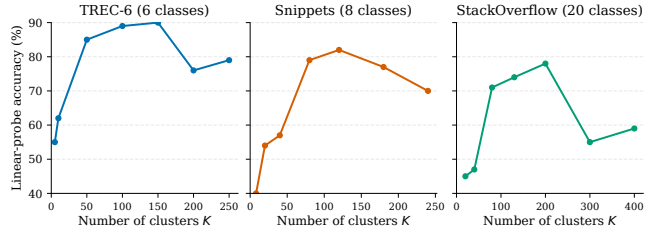


Fig. 2. Validation-split linear-probe accuracy against the number of clusters k . Each point is a separate training run. The k used in Table 2 is the peak of this same validation-split search. The best k exceeds the downstream class count by an order of magnitude on all three tasks.

the best k is roughly an order of magnitude larger. Beyond that point the gains stop and the curves turn back down. Both ends of the range are easy to read. Too few clusters force unrelated topics onto a shared centroid, so a' is minimized by pulling together documents that should stay apart. Too many split each topic into near-duplicates, leaving b' set by a neighbour that differs from the document’s own cluster in name only, and the separation term stops carrying signal. The same preference for large k appears in clustering-based self-supervision for vision and graphs [11, 10], which suggests the clusters work as a partition of the representation space rather than a stand-in for the label set. In practice the region around each peak is broad, so k only needs to be set to the right order of magnitude.

4. LIMITATIONS AND FUTURE WORK

We used a small corpus ($\sim 18.8k$ documents) for clustering in our experiments. On a much larger corpus mini-batch or streaming k -means [20] would take its place. The centroids would then be approximate and whether the gains survive that approximation is the first thing to test. The method also depends on k , which must be chosen per corpus. Finally, the objective may be applied beyond text modality. The same recipe should carry over to images and other modalities. We leave that for future work.

5. CONCLUSION

We presented SILK in this paper, which fine-tunes a language encoder using a Cluster Validation Index as its training loss. Each document is compared against cluster centroids alone and a single optimizer step per epoch is enough. To run it on a Transformer we clustered at the epoch level and accumulated gradients across batches, which keeps memory bounded while every update uses one consistent clustering. SILK trains faster and uses less memory than every baseline we evaluate and stays competitive on linear-probe accuracy. Cluster validation indices have long been used to judge representations; these results suggest they can also be used to learn them.

Compliance with Ethical Standards. This work used publicly available benchmark datasets containing no personally identifiable information, and required no human or animal subjects. The authors declare no conflicts of interest.

6. REFERENCES

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al., “Retrieval-augmented generation for knowledge-intensive nlp tasks,” *Advances in neural information processing systems*, vol. 33, pp. 9459–9474, 2020.
- [2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, 2019, pp. 4171–4186.
- [3] Kawin Ethayarajh, “How contextual are contextualized word representations? comparing the geometry of bert, elmo, and gpt-2 embeddings,” in *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, 2019, pp. 55–65.
- [4] Tianyu Gao, Xingcheng Yao, and Danqi Chen, “Simcse: Simple contrastive learning of sentence embeddings,” in *Proceedings of the 2021 conference on empirical methods in natural language processing*, 2021, pp. 6894–6910.
- [5] Yung-Sung Chuang, Rumen Dangovski, Hongyin Luo, Yang Zhang, Shiyu Chang, Marin Soljačić, Shang-Wen Li, Wen-tau Yih, Yoon Kim, and James Glass, “Diffcse: Difference-based contrastive learning for sentence embeddings,” in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2022, pp. 4207–4218.
- [6] Xing Wu, Chaochen Gao, Liangjun Zang, Jizhong Han, Zhongyuan Wang, and Songlin Hu, “Esimcse: Enhanced sample building method for contrastive learning of unsupervised sentence embedding,” in *Proceedings of the 29th International Conference on Computational Linguistics*, 2022, pp. 3898–3907.
- [7] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Guo, Mohammad Gheshlaghi Azar, et al., “Bootstrap your own latent: a new approach to self-supervised learning,” *Advances in neural information processing systems*, vol. 33, pp. 21271–21284, 2020.
- [8] Tassilo Klein and Moin Nabi, “Scd: Self-contrastive decorrelation of sentence embeddings,” in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022, pp. 394–400.
- [9] Peter J Rousseeuw, “Silhouettes: a graphical aid to the interpretation and validation of cluster analysis,” *Journal of computational and applied mathematics*, vol. 20, pp. 53–65, 1987.
- [10] William Shiao, Uday Singh Saini, Yozen Liu, Tong Zhao, Neil Shah, and Evangelos E Papalexakis, “Carl-g: Clustering-accelerated representation learning on graphs,” in *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023, pp. 2036–2048.
- [11] Mathilde Caron, Piotr Bojanowski, Armand Joulin, and Matthijs Douze, “Deep clustering for unsupervised learning of visual features,” in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 132–149.
- [12] Eduardo R. Hruschka, Leandro N. de Castro, and Ricardo J. G. B. Campello, “Evolutionary algorithms for clustering gene-expression data,” in *Fourth IEEE International Conference on Data Mining (ICDM’04)*. 2004, pp. 403–406, IEEE.
- [13] Jeff Johnson, Matthijs Douze, and Hervé Jégou, “Billion-scale similarity search with gpus,” *IEEE transactions on big data*, vol. 7, no. 3, pp. 535–547, 2019.
- [14] David Arthur, Sergei Vassilvitskii, et al., “k-means++: The advantages of careful seeding,” in *Soda*, 2007, vol. 7, pp. 1027–1035.
- [15] Ken Lang, “Newsweeder: Learning to filter netnews,” in *Machine learning proceedings 1995*, pp. 331–339. Elsevier, 1995.
- [16] Xin Li and Dan Roth, “Learning question classifiers,” in *Coling 2002: The 19th international conference on computational linguistics*, 2002.
- [17] Xuan-Hieu Phan, Le-Minh Nguyen, and Susumu Horiguchi, “Learning to classify short and sparse text & web with hidden topics from large-scale data collections,” in *Proceedings of the 17th international conference on World Wide Web*, 2008, pp. 91–100.
- [18] Jiaming Xu, Peng Wang, Guanhua Tian, Bo Xu, Jun Zhao, Fangyuan Wang, and Hongwei Hao, “Short text clustering via convolutional neural networks,” in *Proceedings of the 1st workshop on vector space modeling for natural language processing*, 2015, pp. 62–69.
- [19] Aaron van den Oord, Yazhe Li, and Oriol Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [20] David Sculley, “Web-scale k-means clustering,” in *Proceedings of the 19th international conference on World wide web*, 2010, pp. 1177–1178.